Universidad Técnica Nacional

Sede San Carlos

Mini Laboratorio

Calidad del Software

Estudiantes:

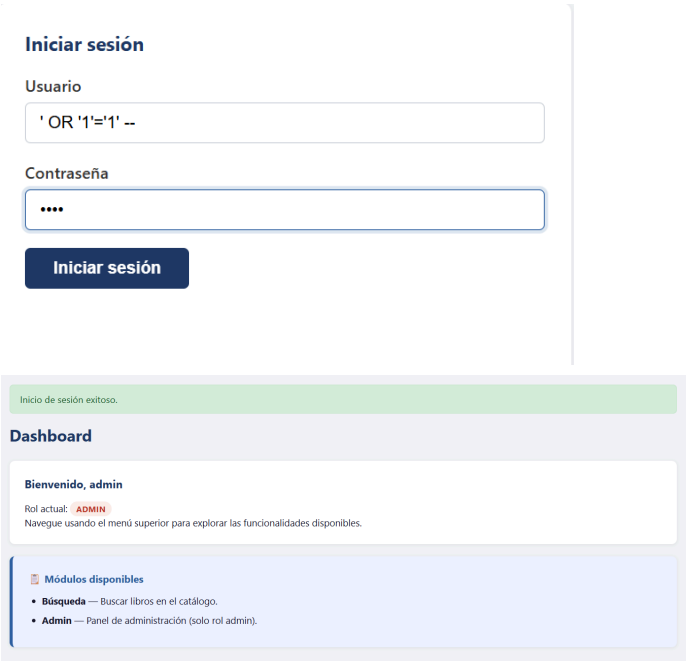
Stiven Arguello Jimenez

Juan Diego Miranda Ureña

Anthony Sancho Badilla

23 de marzo de 2026

Problema #1

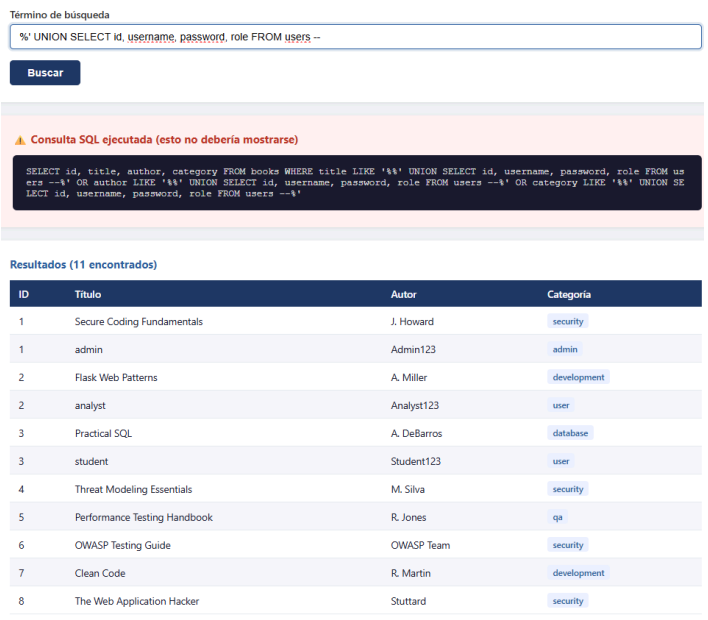
Descripción del problema	La aplicación presenta una vulnerabilidad de **SQL Injection** en el proceso de autenticación. Esto ocurre porque los datos ingresados por el usuario se incorporan directamente en la consulta SQL sin validación ni uso de mecanismos seguros de parametrización. Como resultado, un atacante puede manipular la estructura de la consulta mediante entradas maliciosas.
Evidencia de explotación	
Explicación de por qué es peligrosa	Esta vulnerabilidad permite evadir el proceso de autenticación y obtener acceso no autorizado al sistema. Adicionalmente, puede ser utilizada para acceder, modificar o eliminar información sensible dentro de la base de datos, así como ejecutar consultas arbitrarias. El riesgo se incrementa debido a la exposición de errores internos del sistema, lo que facilita la

	explotación. En conjunto, compromete la confidencialidad, integridad y disponibilidad de la información.

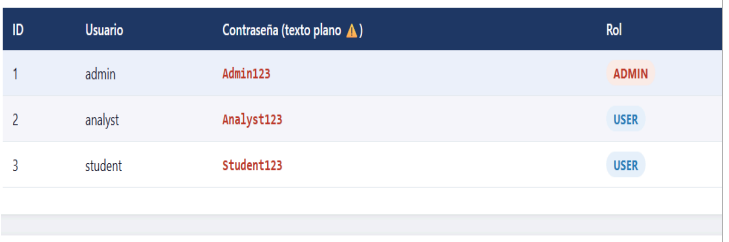
Descripción de la corrección aplicada con fragmento de código antes/después.	Se elimina la siguiente linea de app.py:
	`query = f"SELECT id, username, role FROM users WHERE username = '{username}' AND password = '{password}'"`
	
	Y se realiza la siguiente implementación:
	`conn = get_connection()`
	`try:`
	`user = conn.execute(`
	`"SELECT id, username, role FROM users WHERE username = ? AND password = ?",`
	`(username, password)`
	`).fetchone()`
	`except Exception:`
	`user = None`
	`conn.close()`
	Fix SQL Injection en login reemplazando f-string por consulta parametrizada y eliminando exposición de query

Problema #2

Descripción del problema	Vulnerabilidad de tipo SQL Injection en la funcionalidad de búsqueda de libros. El parámetro term es concatenado directamente dentro de una consulta SQL que utiliza LIKE, sin validación ni parametrización. Esto permite a un atacante alterar la estructura de la consulta y combinar resultados de otras tablas mediante el uso de UNION.

Evidencia de explotación	
Explicación de por qué es peligrosa	Filtración de datos sensibles: permite obtener usuarios y contraseñas directamente. Compromiso total de cuentas: al estar en texto plano, no hay ni siquiera barrera criptográfica. Movimiento lateral: esas credenciales pueden utilizarse.
Descripción de la corrección aplicada con fragmento de código antes/después.	Se elimina la concatenación directa del parámetro `term` dentro de la consulta SQL, la cual permitía la manipulación de la misma mediante entradas maliciosas. Antes: query = f"SELECT id, title, author FROM books WHERE title LIKE '%{term}%'" results = conn.execute(query).fetchall() Despues: conn = get_connection() try: results = conn.execute("SELECT id, title, author FROM books WHERE title LIKE ?", ('%' + term + '%',)).fetchall() except Exception: results = [] conn.close()

Problema #3

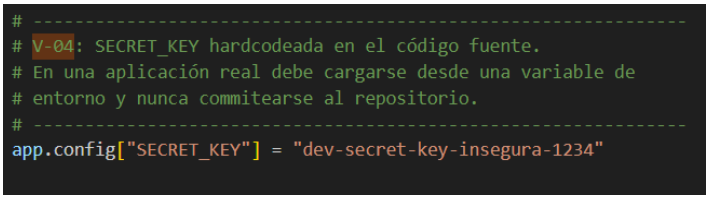
Descripción del problema	La aplicación presenta una vulnerabilidad al almacenar contraseñas en texto plano, es decir, sin aplicar ningún mecanismo de cifrado o hashing seguro. Esto implica que las credenciales de los usuarios se guardan directamente en la base de datos en un formato legible, sin protección frente a accesos no autorizados.
Evidencia de explotación	 ID | Usuario | Contraseña (texto plano ⚠) | Rol 1 | admin | Admin123 | ADMIN 2 | analyst | Analyst123 | USER 3 | student | Student123 | USER
Explicación de por qué es peligrosa	El almacenamiento de contraseñas en texto plano representa un riesgo crítico, ya que cualquier persona con acceso a la base de datos puede visualizar directamente las credenciales de los usuarios. Esto facilita el robo de cuentas y el acceso no autorizado, especialmente considerando que muchos usuarios reutilizan contraseñas en múltiples servicios.

Descripción de la corrección aplicada con fragmento de código antes/después.	Se reemplaza el almacenamiento de contraseñas en texto plano por el uso de hashing seguro utilizando funciones criptográficas.
	
	Antes (inseguro):
	
	`password = request.form["password"]`
	`conn.execute(`
	`  "INSERT INTO users (username, password) VALUES (?, ?)",`
	`  (username, password)`
	`)`

	Después (seguro):
	
	from werkzeug.security import generate_password_hash, check_password_hash
	
	hashed_password = generate_password_hash(password)
	
	conn.execute(
	  "INSERT INTO users (username, password) VALUES (?, ?)",
	  (username, hashed_password)
	)
	
	Y en el login:
	
	user = conn.execute(
	  "SELECT id, username, password, role FROM users WHERE username = ?",
	  (username,)
	).fetchone()
	
	if user and check_password_hash(user["password"], password):
	  # login correcto

Problema #4

Descripción del problema	La aplicación presenta una mala práctica de seguridad al definir la clave secreta directamente en el código fuente (hardcodeada), en lugar de gestionarla de forma segura mediante variables de entorno u otros mecanismos de configuración. Esto implica que la clave queda expuesta en el repositorio o en el código desplegado, aumentando el riesgo de que sea conocida por terceros.

Evidencia de explotación	
Explicación de por qué es peligrosa	La clave secreta es fundamental para la seguridad de la aplicación, ya que se utiliza para proteger sesiones, cookies y otros mecanismos de autenticación. Si un atacante obtiene esta clave, puede manipular o falsificar sesiones, suplantar usuarios y comprometer la integridad de la aplicación.

Descripción de la corrección aplicada con fragmento de código antes/después.	Se crea un archivo .env para guardar estas variables, para que sean más privadas y que no sean accesibles para terceros.
	archivo .env:
	`SECRET_KEY = dev-secret-key-insegura-1234`
	
	Implementacion en app.py:
	`import os`
	`app.config["SECRET_KEY"] = os.getenv("SECRET_KEY")`
	
	Importante, para que esto funcione hay que agregar lo siguiente en el docker-compose:
	`env_file:`
	`  - .env`

Problema #5

Descripción del problema	La aplicación se ejecuta con el modo de depuración activado, lo que habilita funcionalidades de desarrollo que no deben estar presentes en entornos productivos. Este modo expone información interna de la aplicación y activa herramientas avanzadas de

	diagnóstico accesibles desde el navegador.

Evidencia de explotación	
	NameError
	NameError: name 'query' is not defined
	Traceback (most recent call last)
	File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 1498, in __call__
	return self.wsgi_app(environ, start_response)
	File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 1476, in wsgi_app
	response = self.handle_exception(e)
	File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 1473, in wsgi_app
	response = self.full_dispatch_request()
	File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 882, in full_dispatch_request
	rv = self.handle_user_exception(e)
	File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 880, in full_dispatch_request
	rv = self.dispatch_request()
	File "/usr/local/lib/python3.11/site-packages/flask/app.py", line 865, in dispatch_request
	return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
	File "/app/app.py", line 108, in login
	log_event("LOGIN_FAIL", username, f"query={query}")
	NameError: name 'query' is not defined
	Código:
	`if __name__ == "__main__":`
	`init_db()`
	`app.run(host="0.0.0.0", port=5000, debug=True)`

Explicación de por qué es peligrosa	Tener el modo debug activo representa un riesgo crítico, ya que permite a un atacante acceder a información sensible del sistema, como trazas de errores, variables internas y estructura de la aplicación. Además, en ciertos casos, puede habilitar un entorno interactivo que permite la ejecución remota de código en el servidor, comprometiendo completamente la seguridad del sistema.
Descripción de la corrección aplicada con fragmento de código antes/después.	Se desactiva el modo debug en el entorno de ejecución para evitar la exposición de información sensible. **Antes (inseguro):** app.run(host="0.0.0.0", port=5000, debug=True) **Después (seguro):** app.run(host="0.0.0.0", port=5000, debug=False)

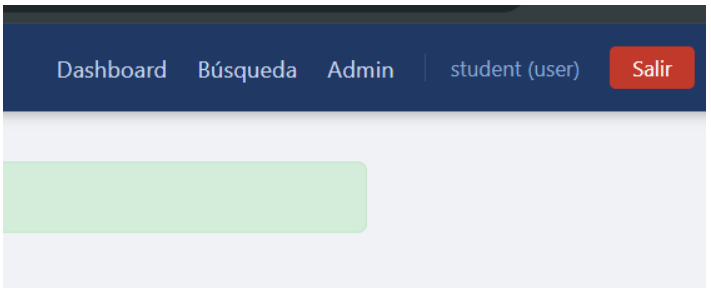
	Se elimina el modo de depuración en producción para prevenir la exposición de trazas internas y la posible ejecución remota de código.

Problema #6

Descripción del problema	La aplicación expone información interna al mostrar directamente consultas SQL en la interfaz de usuario. Esto implica que detalles sobre la estructura de la base de datos, como tablas y campos, quedan visibles para cualquier usuario que interactúe con la funcionalidad.
Evidencia de explotación	
Explicación de por qué es peligrosa	La exposición de consultas SQL facilita la identificación de la estructura interna de la base de datos, lo que puede ser aprovechado por un atacante para desarrollar ataques más precisos, como **SQL Injection**. Además, revela información sensible sobre la lógica de la aplicación, reduciendo el esfuerzo necesario para encontrar y explotar otras vulnerabilidades.
Descripción de la corrección aplicada con fragmento de código antes/después.	Se elimina la visualización directa de consultas SQL en la interfaz de usuario, evitando la exposición de información interna. **Antes (inseguro):** return render_template("search.html", results=results, query=query)

	En la vista: Consulta ejecutada: {{ query }} **Después (seguro):** return render_template("search.html", results=results) Se elimina el envío de la consulta SQL a la vista, evitando que el usuario tenga acceso a la estructura interna de la base de datos.

Problema #7

Descripción del problema	La aplicación presenta una falla en el control de acceso a nivel de interfaz, ya que muestra opciones de navegación (como el acceso a funciones administrativas) a todos los usuarios autenticados, sin considerar su rol o nivel de privilegio. Esto provoca que usuarios sin permisos adecuados puedan visualizar funcionalidades restringidas.
Evidencia de explotación	
Explicación de por qué es peligrosa	Aunque el backend pueda restringir el acceso, exponer estas opciones en la interfaz facilita la enumeración de funcionalidades sensibles y revela la existencia de rutas o módulos administrativos. Esto puede incentivar intentos de acceso no autorizado y facilitar ataques dirigidos, como manipulación de rutas o pruebas de privilegios.

Descripción de la corrección aplicada con fragmento de código antes/después.	Se implementa una validación en el archivo layout.html para verificar roles, si el rol no es admin, la opción admin del navbar no se muestra.
	layout.html:
	`{% if session.get('role') == 'admin' %}`
	`<a href="{{ url_for('admin') }}">Admin</a>`
	`{% endif %}`

Problema #8

Descripción del problema	Ninguna de las rutas que aceptan solicitudes POST (como /login o /search) implementa tokens CSRF para validar que la petición proviene realmente del usuario legítimo y no de un sitio externo malicioso.
Evidencia de explotación	<form action="http://localhost:5000/search" method="POST"> <input type="hidden" name="term" value="test">> </form> <script> document.forms[0].submit(); </script>
Explicación de por qué es peligrosa	- Ejecución de acciones sin consentimiento. - Encadenamiento con otras vulnerabilidades: CSRF + SQL Injection = ataque automatizado - Robo indirecto de información - Abuso de sesiones activas
Descripción de la corrección aplicada con fragmento de código antes/después.	Se implementa protección contra ataques CSRF mediante el uso de tokens generados automáticamente en los formularios. **Antes (inseguro):** <form method="POST" action="/search">> <input type="text" name="term">> <button type="submit">Buscar</button> </form> **Después (seguro):** from flask_wtf import CSRFProtect csrf = CSRFProtect(app) **En los formularios:** <form method="POST" action="/search">> <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">> <input type="text" name="term">>

	<button type="submit">Buscar</button> </form> Se incorpora un token CSRF en cada formulario para validar la autenticidad de las solicitudes y prevenir ejecuciones no autorizadas desde sitios externos.